Tarefa 18: Soma de Vetores em CPU e GPU com OpenMP

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

O objetivo da atividade é executar a adição de vetores (`vadd.c`) em um nó com GPU do NPAD (partição `gpu-4-h200`) e comparar o tempo de execução usando somente a CPU com o tempo usando a GPU, por meio da diretiva `#pragma omp target` do OpenMP.

A operação é $c[i] = a[i] + b[i]$: cada elemento é independente, o que a torna ideal para paralelismo de dados, mas faz apenas uma soma para cada 12 bytes acessados (2 leituras e 1 escrita de `float`). Ela é, portanto, limitada pela largura de banda de memória, não pela capacidade de cálculo.

2 Metodologia

O código foi separado em duas funções, `soma_cpu` e `soma_gpu` (Apêndice A):

- **CPU:** `#pragma omp parallel for`, usando todas as threads do nó.
- **GPU:** `#pragma omp target parallel for` com `map(to: a, b)` e `map(from: c)`. O tempo medido **inclui** a cópia dos dados entre CPU e GPU.

O tamanho N cresce de 10^4 a $2 \cdot 10^8$ elementos, com 5 repetições por tamanho. Após cada soma, o resultado é verificado contra o valor esperado ($6i$). Os tempos foram medidos com `omp_get_wtime` e salvos em `resultados.csv`. Nas estatísticas foi usada a **mediana das repetições 2 a 5**, descartando a primeira, que sofre efeitos de inicialização (*page faults* e início do *runtime*).

3 Resultados

Nenhuma execução apresentou erros de verificação (`erros_cpu` e `erros_gpu` iguais a 0). A Tabela 1 mostra as medianas e a Figura 1 os mesmos dados em escala logarítmica.

4 Análise

A CPU foi mais rápida que a GPU em todos os tamanhos testados. O motivo é que, nessa operação, o tempo da GPU é gasto quase todo movendo dados, não calculando:

N	CPU (ms)	GPU (ms)	Speedup	Vazão GPU (GB/s)
10^4	0,005	0,42	0,012	0,3
10^5	0,007	0,51	0,014	2,4
10^6	0,032	0,98	0,032	12,2
$5 \cdot 10^6$	0,313	3,43	0,091	17,5
10^7	0,746	6,46	0,115	18,6
$5 \cdot 10^7$	5,00	17,45	0,286	34,4
10^8	10,42	22,51	0,463	53,3
$2 \cdot 10^8$	21,41	45,59	0,470	52,6

Tabela 1: Tempos medianos (repetições 2 a 5). Speedup $= t_{CPU}/t_{GPU}$; vazão $= 12N$ bytes / t_{GPU}.

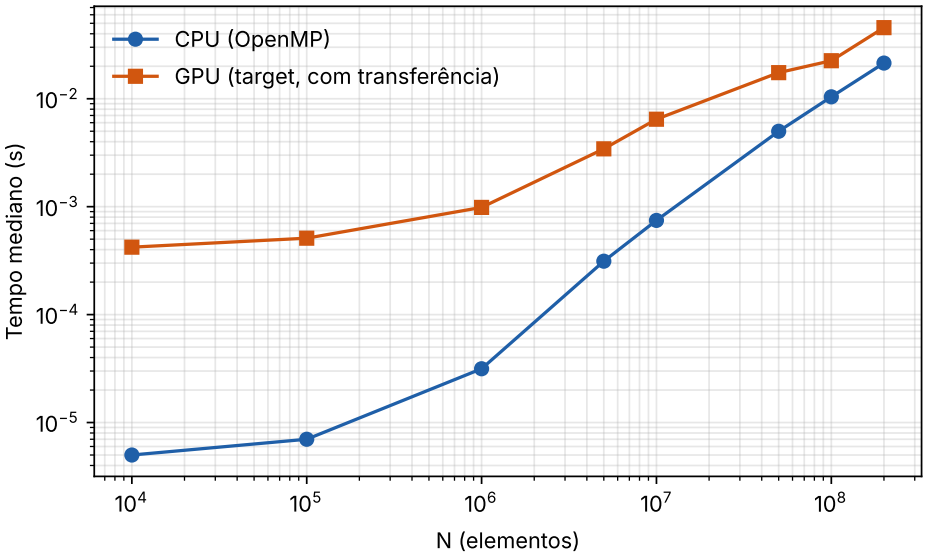


Figura 1: Tempo de execução em função de N (escala log-log).

- **Pouco cálculo por dado:** cada elemento exige apenas uma soma para 12 bytes acessados. A GPU é muito rápida em cálculo, mas aqui não há cálculo suficiente para aproveitar essa vantagem.
- **Cópia entre CPU e GPU:** os vetores a e b precisam ser enviados à GPU e c trazido de volta, por um barramento muito mais lento que a memória. Para $N \geq 10^8$ a vazão efetiva da GPU é de ≈53 GB/s, o que mostra que o tempo é o da cópia. A CPU já tem os dados na sua própria memória e não paga esse custo.
- **Custo fixo de lançamento:** cerca de 0,4 a 0,5 ms por chamada, mesmo com $N = 10^4$. A CPU resolve esse mesmo tamanho em microssegundos.

Com o aumento de N a diferença diminui (speedup de 0,01 para 0,47), pois o custo fixo passa a pesar menos, mas a GPU continua mais lenta, porque a cópia cresce junto com N.

A Código-Fonte

```c
/*
 * Tarefa 18 - Adição de vetores: CPU (OpenMP) x GPU (OpenMP target)
 *
 * Compilação (exemplos):
 *   GCC (nvptx)   : gcc -fopenmp -foffload=nvptx-none -O3 tarefa18.c -o tarefa18
 *   Sem GPU       : gcc -fopenmp -O3 tarefa18.c -o tarefa18
 *
 * Uso: ./tarefa18 [N_max] [repeticoes]
 * Saída: resultados.csv
 */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define REL_TOL 1e-6f

/* Soma de vetores somente na CPU */
void soma_cpu(const float *a, const float *b, float *c, long n)
{
    #pragma omp parallel for
    for (long i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}

/* Soma de vetores na GPU */
void soma_gpu(const float *a, const float *b, float *c, long n)
{
    #pragma omp target parallel for map(to: a[0:n], b[0:n]) map(from: c[0:n])
    for (long i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}

/* Conta elementos com erro relativo acima da tolerância (esperado: 6*i) */
long verifica(const float *c, long n)
{
    long erros = 0;
    #pragma omp parallel for reduction(+:erros)
    for (long i = 0; i < n; i++) {
        float esperado = 6.0f * (float)i;
        if (fabsf(c[i] - esperado) > REL_TOL * fabsf(esperado)) erros++;
    }
    return erros;
}

int main(int argc, char **argv)
{
    long n_max = (argc > 1) ? atol(argv[1]) : 100000000L;
    int  reps  = (argc > 2) ? atoi(argv[2]) : 5;

    long tamanhos[] = {10000L, 100000L, 1000000L, 5000000L, 10000000L,
                       50000000L, 100000000L, 200000000L};
    int  ntam = sizeof(tamanhos) / sizeof(tamanhos[0]);

    printf("Threads CPU: %d | Dispositivos GPU: %d\n",
           omp_get_max_threads(), omp_get_num_devices());
```

```c

    FILE *csv = fopen("resultados.csv", "w");
    if (!csv) { perror("resultados.csv"); return 1; }
    fprintf(csv, "N,repeticao,tempo_cpu_s,tempo_gpu_s,speedup,erros_cpu,erros_gpu\n");

    for (int t = 0; t < ntam; t++) {
        long n = tamanhos[t];
        if (n > n_max) break;

        float *a = malloc(n * sizeof(float));
        float *b = malloc(n * sizeof(float));
        float *c = malloc(n * sizeof(float));
        if (!a || !b || !c) {
            fprintf(stderr, "Sem memória para N=%ld\n", n);
            free(a); free(b); free(c);
            break;
        }

        #pragma omp parallel for
        for (long i = 0; i < n; i++) {
            a[i] = (float)i;
            b[i] = 5.0f * (float)i;
        }

        for (int r = 1; r <= reps; r++) {
            double t0, t_cpu, t_gpu;

            t0 = omp_get_wtime();
            soma_cpu(a, b, c, n);
            t_cpu = omp_get_wtime() - t0;
            long err_cpu = verifica(c, n);

            #pragma omp parallel for
            for (long i = 0; i < n; i++) c[i] = 0.0f;

            t0 = omp_get_wtime();
            soma_gpu(a, b, c, n);
            t_gpu = omp_get_wtime() - t0;
            long err_gpu = verifica(c, n);

            fprintf(csv, "%ld,%d,%.6f,%.6f,%.3f,%ld,%ld\n",
                    n, r, t_cpu, t_gpu, t_cpu / t_gpu, err_cpu, err_gpu);
            printf("N=%-10ld rep=%d CPU=%.5fs GPU=%.5fs erros=%ld/%ld\n",
                   n, r, t_cpu, t_gpu, err_cpu, err_gpu);
        }
        free(a); free(b); free(c);
    }

    fclose(csv);
    printf("Dados salvos em resultados.csv\n");
    return 0;
}
```

Código 1: tarefa18.c